

Work Contribution — Jenith Gurung

Student ID: 25068608

Overview

My responsibility in the group was the backend foundation — the login system, file handling, and main program flow. This work was completed first since all other modules (admin.py, student.py, analytics.py) depend on it for reading and writing data.

Files Delivered

file_handler.py

Handles all read and write operations across the four data files required by the brief. The module provides consistent functions for each file:

- read_users() and write_users() — parses and writes users.txt with four fields: id, name, role, username
- read_passwords(), write_passwords(), check_password(), add_password() — validates and manages login credentials
- read_grades() and get_grades() — loads subject marks for analytics and student view
- read_eca() and get_eca() — loads extracurricular activity records

All file operations use try/except blocks to handle FileNotFoundError and other exceptions gracefully, satisfying the brief's exception handling requirement.

auth.py

Implements the login system and OOP structure required by the brief. Key components:

- User base class — stores user_id, name, role, username with is_admin() and is_student() methods
- Student subclass — inherits from User, includes get_grades() method
- Admin subclass — inherits from User
- login() function — validates username and password, enforces a three-attempt lockout, and returns the correct subclass instance based on role
- logout() function — clears the current session

The inheritance structure between User, Student, and Admin directly satisfies the brief's OOP and inheritance requirement.

main.py

Entry point for the full system. Responsibilities:

- Calls login() from auth.py and handles failed login by exiting
- Routes admin users to admin_control() in admin.py
- Routes student users to the student menu, calling view_profile(), view_grades(), view_eca(), update_profile() from student.py
- Uses try/except ImportError so the program does not crash if a teammate's file is missing
- Calls logout() on exit regardless of which path was taken

Data Format Design

I designed the format for all four .txt files used by the system. Each file uses a comma-separated format with a header row on the first line:

File	Format
users.txt	id, name, role, username
passwords.txt	username, password
grades.txt	id, math, science, english, cs, stats
eca.txt	id, activity

A key design decision was adding a username column to users.txt. The original team plan used only three columns (id, name, role), but this made it impossible to map a login username to a user record. Adding username as a fourth column resolved this cleanly and I communicated the change to all team members.

Integration Contribution

Beyond my own three files, I also helped resolve integration issues on the day of submission. I identified that student.py was importing functions that did not exist in file_handler.py, and that admin.py was running its menu loop at module level rather than inside a callable function. I guided the team through the fixes via the group chat and updated main.py to call admin_control() directly once Abijeet confirmed his function name.